

fairly straightforward—as you saw in [Chapter 14](#), “[Graphs](#),”—for the algorithm to choose one edge to add to the minimum spanning tree. When edges have different weights, however, you need to choose a bit more carefully.

An Example: Networking in the Jungle

Suppose you want to install high-speed network lines to connect six cities in the mythical country of Turala. Five links are all that is needed to connect the six cities, but which five links should they be? The cost of connecting each pair of cities varies, so you must pick the route carefully to minimize the overall cost. [Figure 15-1](#) shows a weighted graph with six vertices. Each edge has a weight, shown by a number alongside the edge. This is the abstract form of a weighed graph. Notice that some links are not shown (for example, no direct link from A to D or A to F). When the graph represents a real-world problem, some links could be impractical because of distance, terrain, environmental, or other issues.

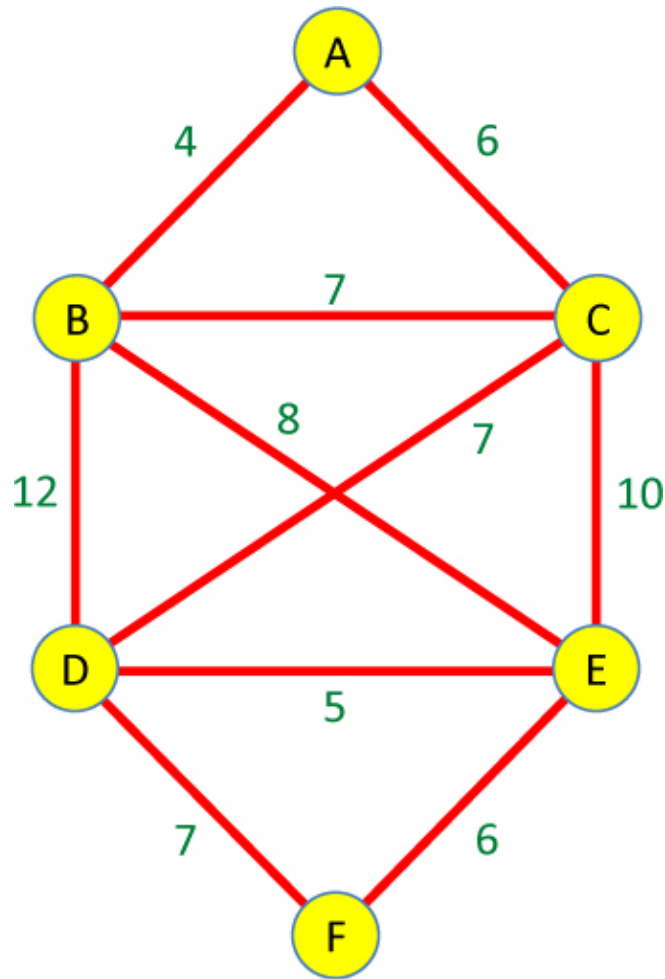


Figure 15-1 *A weighted graph*

How can you pick a subgraph that minimizes the cost of connecting vertices into a network? The answer is to calculate a minimum spanning tree. It will have five edges (one fewer than the number of vertices), it will connect all six cities, and it will minimize the total cost of the links. Can you figure out this route by looking at the graph in [Figure 15-1](#)? If not, you can solve the problem with the WeightedGraph Visualization tool.

The WeightedGraph Visualization Tool

The WeightedGraph Visualization tool is like the Graph tool, but it creates weighted, undirected graphs. You can create vertices, link them with edges, and show or hide the adjacency matrix them as before. In the shaded graph region, you can double-click to create a new vertex, drag from one vertex to another to

create an edge, and drag using the second mouse button or by holding the Shift key to move a vertex. Double-clicking an edge or vertex deletes it.

Edges created by dragging get an initial weight of 1. You can change the weight by selecting it—either in the shaded graph region or in the adjacency matrix—and changing the numeric value. As you change weights in one place, they update everywhere. If you erase the weight, or make it zero, the edge disappears. Weights are restricted to the values 1–99.

Try out this tool by creating some small graphs and finding their minimum spanning trees. (For some configurations, you’ll need to be careful positioning the vertices so that the weight numbers don’t fall on top of each other.)

Use the WeightedGraph Visualization tool to construct the graph of [Figure 15-1](#). The result should look something like [Figure 15-2](#). If you happen to have the visualization tool running on a computer where you can launch it from the command line, you can initialize the graph (with random positioning of the vertices) using this command:

```
python3 WeightedGraph.py A B C D E F A-B:4 A-C:6 B-C:7 B-D:12 B-E:8 C-  
D:7  
C-E:10 D-E:5 D-F:7 E-F:6
```

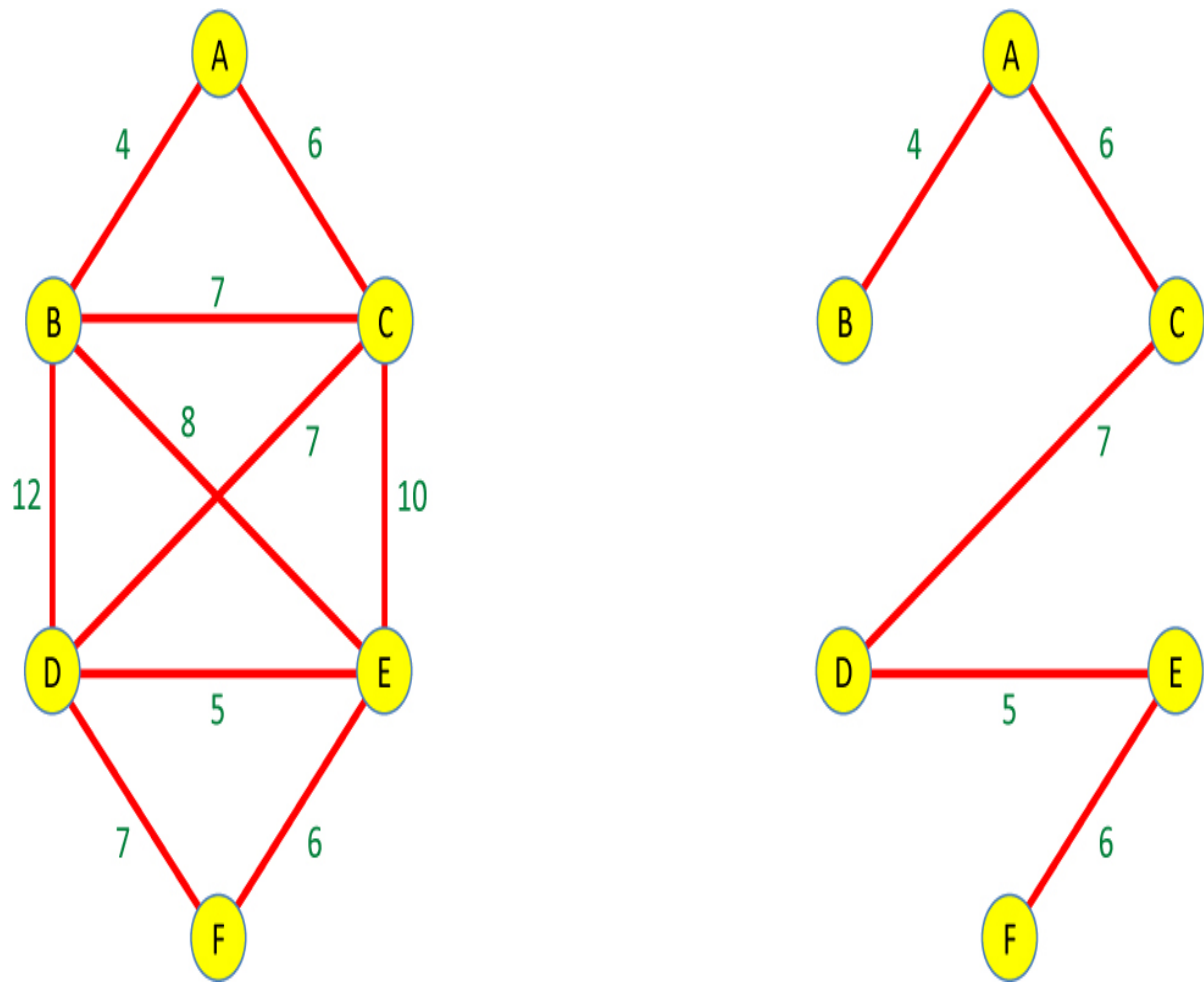


Figure 15-2 *The WeightedGraph Visualization tool*

Now find this graph's minimum spanning tree starting from vertex A by clicking that vertex to highlight it with the blue ring and then selecting the Minimum Spanning Tree button. The result should be the minimum spanning tree shown in [Figure 15-3](#).

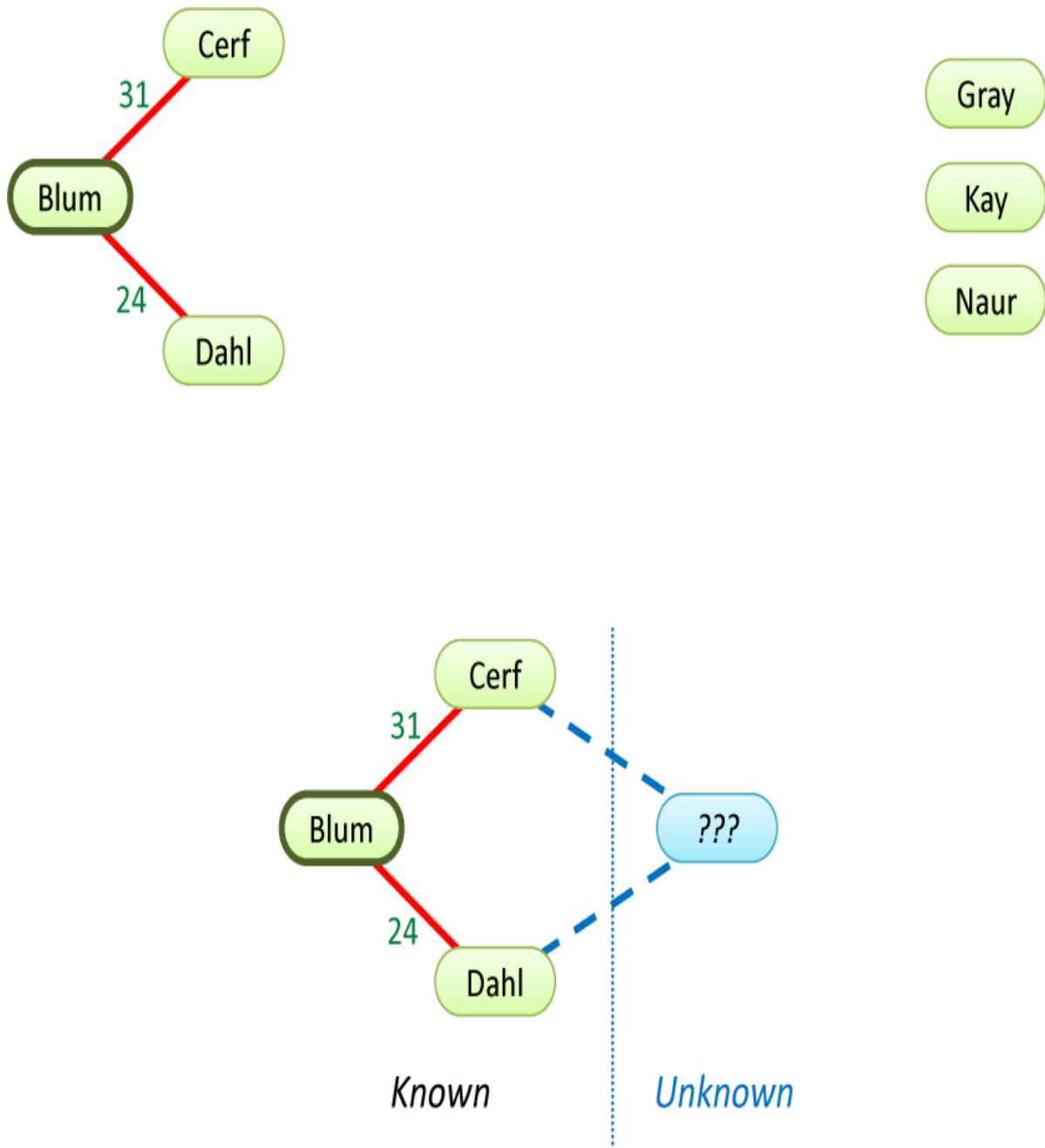


Figure 15-3 *The minimum spanning tree of the previous graph*

The tool discovers that the minimum spanning tree consists of the edges AB, AC, CD, DE, and EF, for a total edge weight of 28. The order in which the edges are specified is unimportant. If you start at a different vertex, you will create a tree with the same edges, but in a different order. For a graph with distinct connected components, the minimum spanning tree connects only one of them, as you saw in [Chapter 14](#).

Building the Minimum Spanning Tree: Send Out the Surveyors

The algorithm for constructing the minimum spanning tree is a little involved, so we're going to introduce it with the analogy of building a high-speed network in Turala. Imagine that your company won the contract to bring high-speed networking to this jungle-covered country. You are the project manager, of course, and there are also various surveyors on your team.

The first challenge of this project is something that happens in many problems: you have to “discover” the graph. Although the location of the cities and the approximate cost of laying network cables or creating point-to-point microwave transmission systems are known, the exact details of the terrain between the cities is not. That means that the graph and its edge weights are unknown at the start. Part of your job is to discover that information.

Computer algorithms always focus on tiny parts of a problem. Unlike humans who like to look at the big picture, computers focus on a single vertex, edge, node, key, and so on, at a time, making a series of local decisions that combine to arrive at a particular goal. With graphs, algorithms tend to start at some vertex and work away from it, acquiring data about nearby vertices before finding out about vertices farther away. This is especially true in large graphs, which are difficult or impossible to fully represent at one time, such as all possible routes between the stars in a galaxy.

In a similar way, planning the route in Turala involves getting to some of the cities and finding out what lies on the roads and paths linking them. Acquiring this information takes time. That's where your surveyors come in.

Starting in Blum

You start by setting up an office in the city of Blum. (You could start in any city, but you've heard Blum has the best restaurants.) The other cities on the contract to be connected are Cerf, Dahl, Gray, Kay, and Naur. Blum is at one end of the country, and you learn that only two cities can be reached from Blum—Cerf and Dahl—as shown in [Figure 15-4](#). The graph shows Blum with a thicker outline to indicate you have an office there.

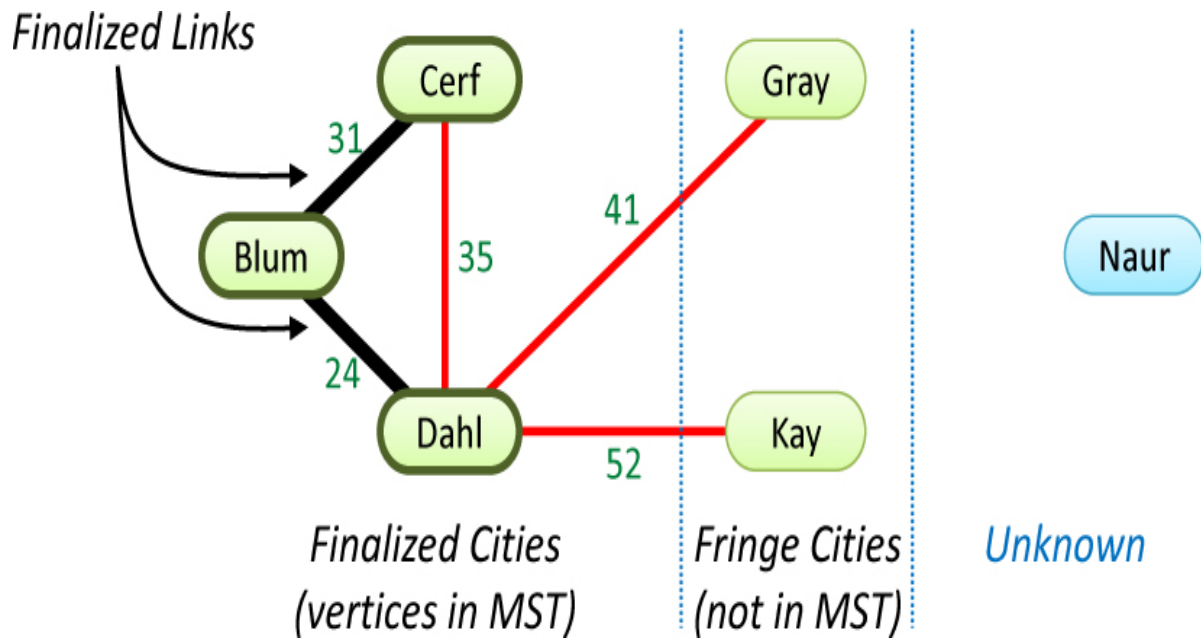


Figure 15-4 *First cities visited in Turala*

Technically, you could try to reach the other cities by direct paths through the jungle. The network must reach those two cities, and your analysis shows that it will be less expensive to go through one or both cities than to cut through the jungle, mountains, and rivers on some new path.

You send two of your surveying teams out along the paths to these cities to see what conditions are like. Their job is to evaluate these routes for different networking options (buried cable, overhead cable, transmission towers, and so on). They take the actual distance and add extra cost factors for soil conditions, river crossings, elevation changes, and the like.

The first team arrives in Cerf and calls you with their report; they say the link from Blum to Cerf should cost 31 million Turala kopeks (and the scenery is gorgeous). The second team reports a little later from Dahl that the Blum–Dahl link, which crosses more level country, should cost 24 million. You make a list:

Blum–Dahl, 24

Blum–Cerf, 31

You always list the links in order of increasing cost; you’ll see why this is a good idea soon.

Finalizing the Blum–Dahl Link

At this point you figure you can send out the construction crew to actually build the Blum–Dahl link. How can you be sure that link will eventually be part of the optimal solution (the minimum spanning tree)? So far, you know the cost of only two links in the system. Don't you need more information?

To get a feel for this situation, try to imagine some other route linking Blum to Dahl that would be better than the direct link. If it doesn't go directly to Dahl, this other route must go through Cerf and circle back to Dahl, possibly via one or more other cities indicated by the question marks in [Figure 15-5](#). You already know the link from Blum to Cerf would be more costly, 31 million, than the link from Blum to Dahl at 24. Your preplanning indicated that trying to reach any of the other cities directly from Blum would be even more costly. So even if the remaining links in this hypothetical circle route are cheap, with a cost of 1 million (or even 0), it will still cost more to get to Dahl from Blum by going through Cerf and the unknown cities (> 31 million). OK, then what if the costs are high on the unknown circle route, say 100 (or really anything bigger than 31)? If those costs are high, you'll probably keep both the Blum–Dahl and Blum–Cerf routes in the plan. In both cases for the unknown costs, the Blum–Dahl route stays. The Blum–Cerf link is not certain, so you won't build anything there yet.

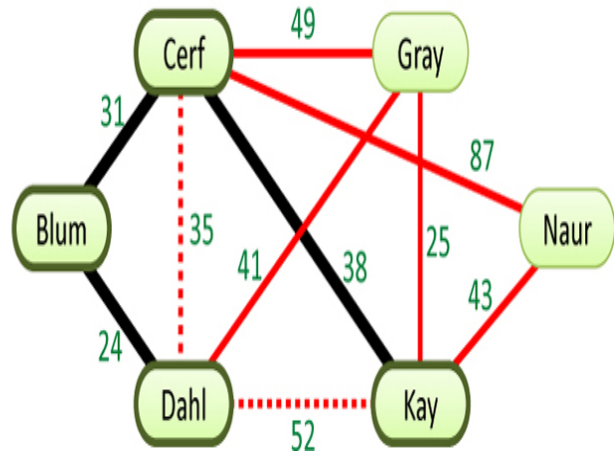
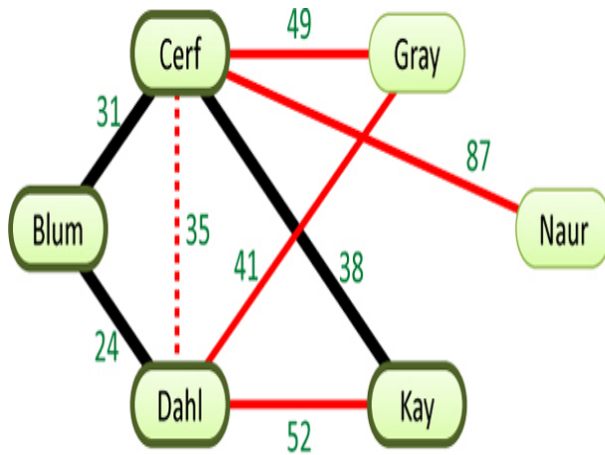


Figure 15-5 *Hypothetical circle route to Dahl*

You conclude that the Blum–Dahl route will be part of the minimum spanning tree. This isn’t a formal proof (which is beyond the scope of this book), but it does suggest your best bet is to pick the cheapest link. You send some of your staff to set up an office in Dahl.

Why do you need another office? Due to a Turalan government regulation, you must install an office before you can send out surveyors from a town. In graph terms, you must add a vertex to the tree before you can learn the weight of the edges leading away from that vertex. All towns with offices are on links that will be in the final minimum spanning tree; towns with no offices are not yet connected.

Building the Blum–Cerf Link

After you've completed the Blum–Dahl link and built your new office, you can send out surveyors from Dahl to all the cities reachable from there. They learn that these are Cerf, Gray, and Kay. The survey teams reach their destinations and report back costs of 35, 41, and 52 million, respectively. Of course, you don't send a survey team back to Blum because you've already surveyed the route, installed the networking link, and have an office there (with a nice view of the coast).

Now you know the costs of four links (in addition to the one you've already built):

- Blum–Cerf, 31
- Dahl–Cerf, 35
- Dahl–Gray, 41
- Dahl–Kay, 52

At this point it may not be obvious what to do next. There are many potential links to choose from. What do you imagine is the best strategy now? Here's the rule:

Rule

From the list, always pick the lowest-cost edge.

Actually, you already followed this rule when you chose which route to follow from Blum; the Blum–Dahl edge had the lowest weight (cost). Here the lowest-cost edge is Blum–Cerf, so you can now finalize the Blum-to-Cerf route at a cost of 31 million. You can now open an office in Cerf (and enjoy the unique musical culture there).

Let's pause for a moment and make a general observation. At a given time in planning the best route, there are three kinds of cities:

1. Cities on routes that have been finalized. (In graph terms they're vertices in the minimum spanning tree.)
2. Cities that have been visited by the surveyors, so you know the cost to link them to at least one city in the first group of cities. You can call these "fringe" cities.
3. Cities that have not been visited by your team.

At this stage, Blum, Dahl, and Cerf are in category 1; Gray and Kay are in category 2; and Naur is in category 3, as shown in [Figure 15-6](#). As you work your way through the algorithm, cities move from unknown to fringe, and from fringe to finalized.

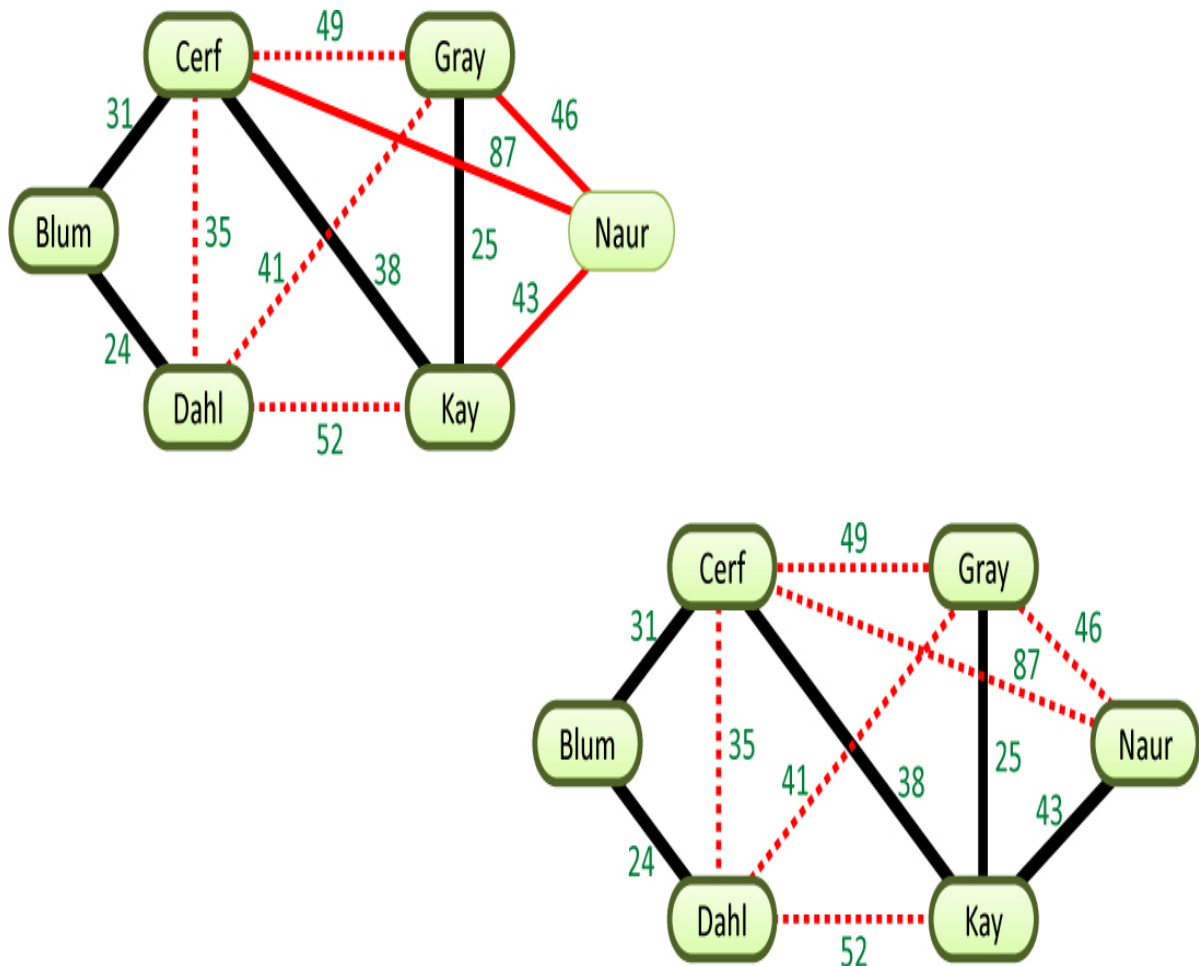


Figure 15-6 *Partway through the minimum spanning tree algorithm*

Building the Cerf–Kay Link

At this point, Blum, Dahl, and Cerf are connected to the network and have offices. You already know the costs from Blum and Dahl to cities in category 2, but you don't know the costs of connecting from Cerf. So, from Cerf you send out surveyors to Gray, Kay, and Naur. They report back costs of 49 million to Gray, 38 million to Kay, and 87 million to Naur. Here's the new list:

~~Dahl–Cerf, 35~~

Cerf–Kay, 38

Dahl–Gray, 41

Cerf–Gray, 49

Dahl–Kay, 52

Cerf–Naur, 87

The Dahl–Cerf link was on the previous list and is crossed out in this list. Why? Well, there’s no point in considering links to cities that are already connected, even by an indirect route. Furthermore, minimum spanning trees must not contain cycles. You can now revise your rule slightly:

Rule

From the list, always pick the lowest-cost edge *to a fringe city (vertex)*.

From this list, you can see that the next least costly route is Cerf–Kay, at 38 million. You send out the crew to install this link and set up an office in Kay, resulting in the graph of [Figure 15-7](#).

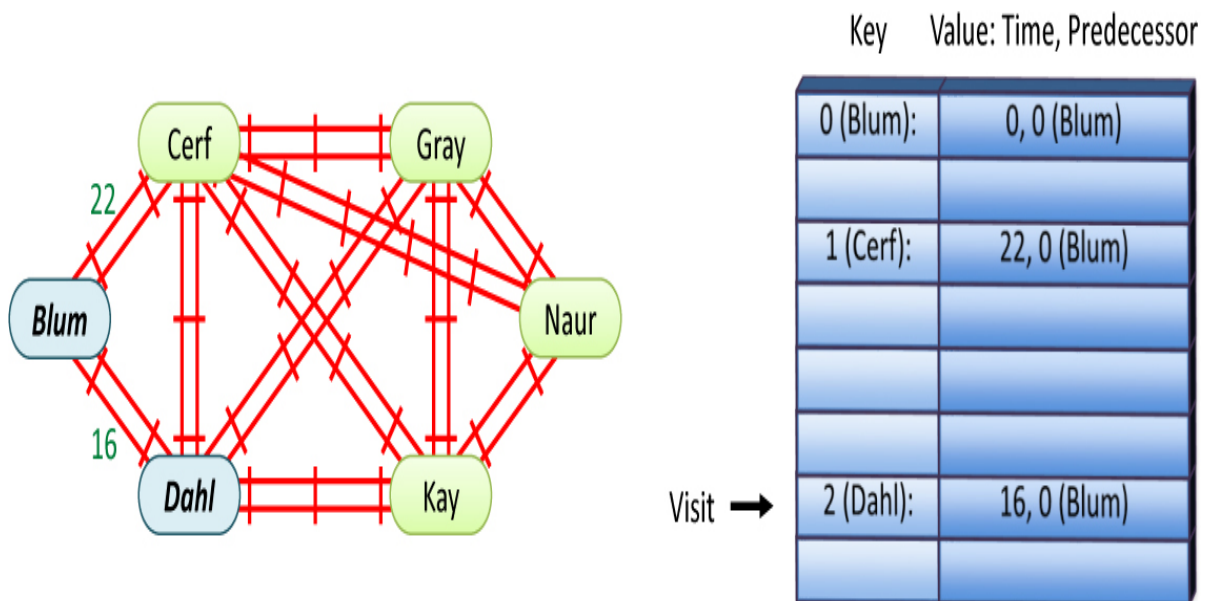
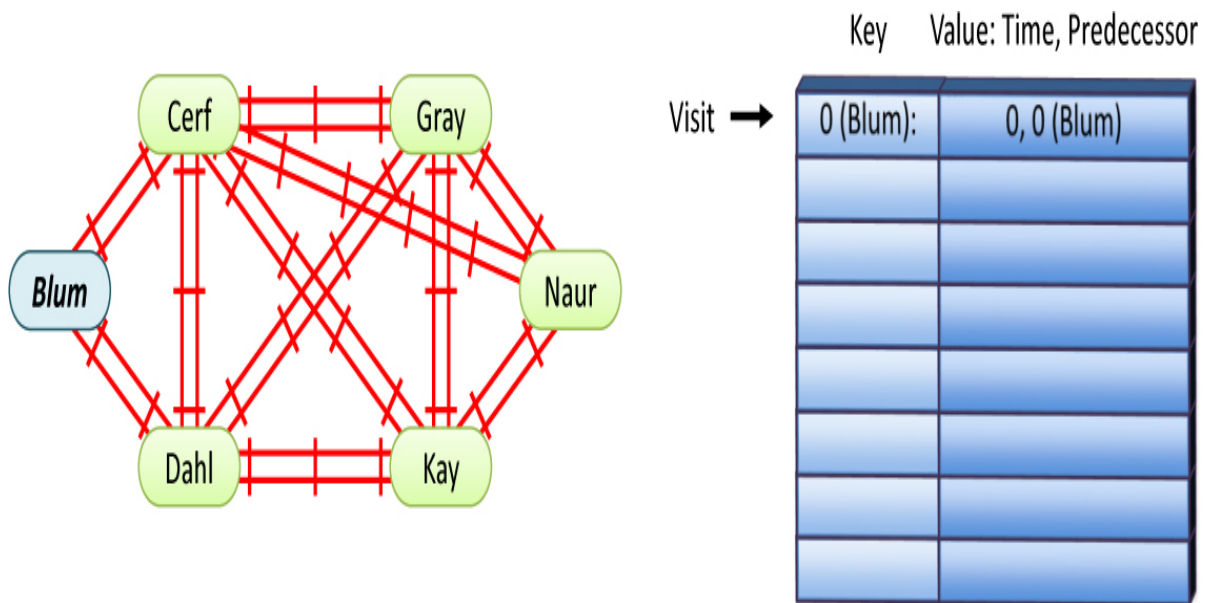


Figure 15-7 *The minimum spanning tree after visiting all links from Cerf*

Building the Kay–Gray Link

From Kay, you dispatch more survey teams, and they report back costs of 25 million to Gray and 40 to Naur. The Dahl–Kay link from the previous list must be removed because Kay is now a connected city. Your new list of edges to fringe cities (ignoring the crossed out links to finalized cities) is

Kay–Gray, 25

Dahl–Gray, 41

Kay–Naur, 43

Cerf–Gray, 49

Cerf–Naur, 87

The lowest-cost link is Kay–Gray, so you build this link and install an office in Gray.

And, Finally, the Link to Naur

The choices are narrowing. After you remove already-linked cities and send out surveyors from Gray, the situation appears like the left-hand graph in [Figure 15-8](#). Your list now shows only:

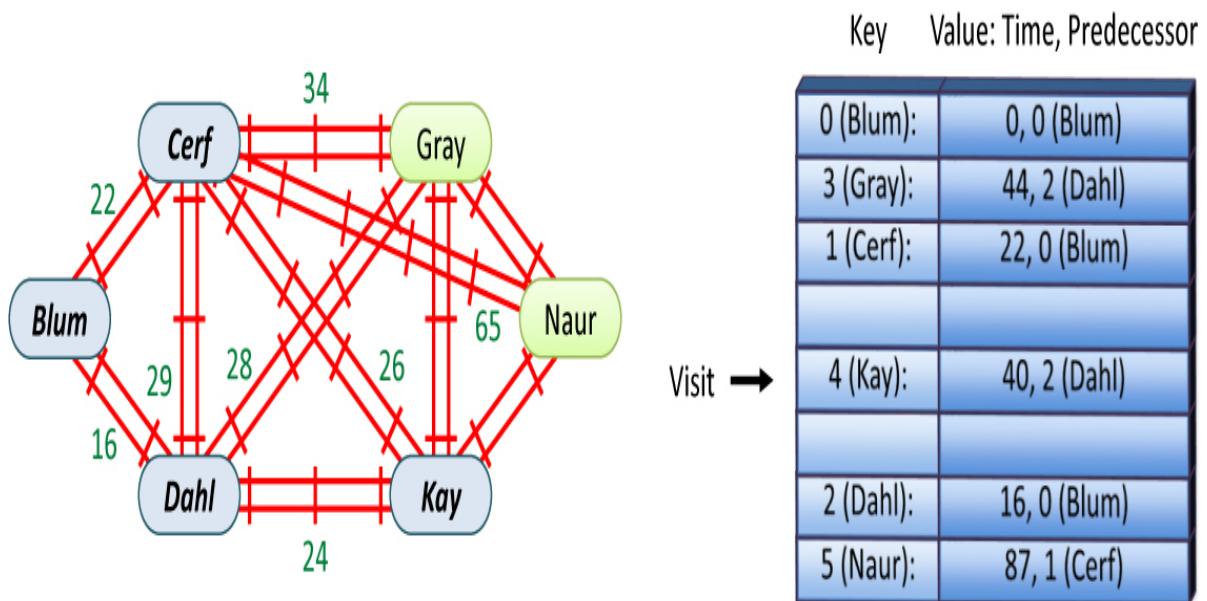
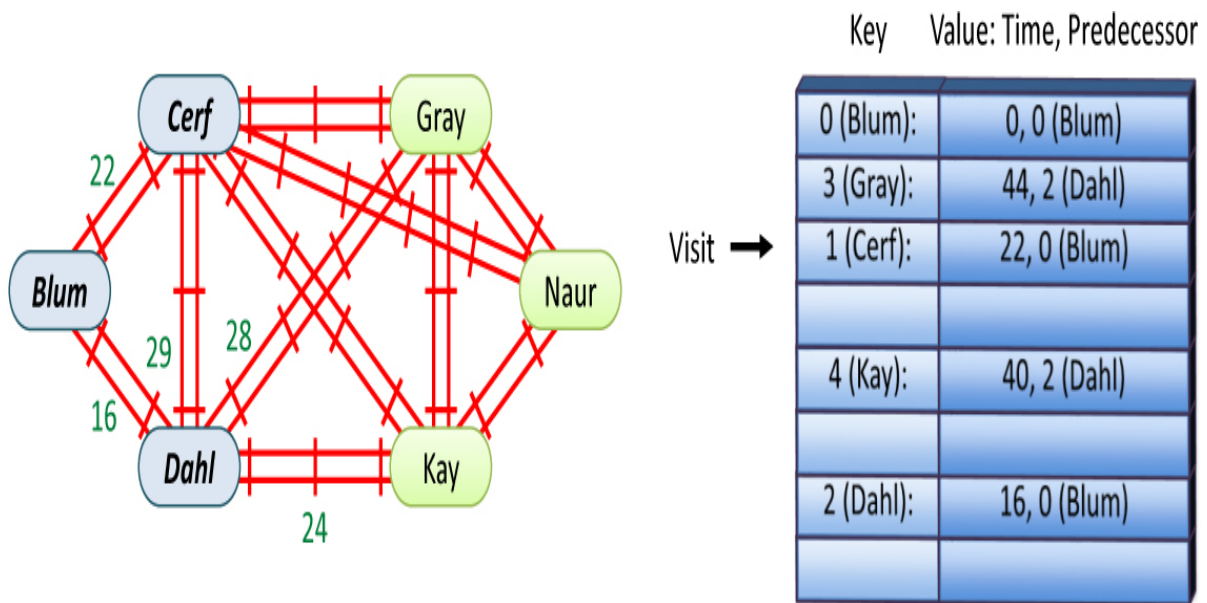


Figure 15-8 *The final connections of the minimum spanning tree in Turala*

Kay–Naur, 43

Gray–Naur, 46

Cerf–Naur, 87

You install the last link from Kay to Naur, build an office in Naur, and you're done. You know you're done because there's now an office in all six cities.

You've constructed the links for Blum–Dahl, Blum–Cerf, Cerf–Kay, Kay–Gray, and Kay–Naur, as shown on the right of [Figure 15-8](#). This is the lowest-cost network at 161 million ($24 + 31 + 38 + 25 + 43$) linking the six cities in Turala.

Note that the final minimum spanning tree does not include all of the lowest-cost edges. The last link added, from Kay to Naur, had a cost of 43. That's higher than the cost of Dahl–Cerf at 35 and Dahl–Gray at 41. Could there be another minimum spanning tree that uses one of those links? The short answer is yes; there can be other spanning trees with an equal total cost (but none with a lower cost).

Creating the Algorithm

Using the somewhat fanciful idea of networking a small country, we've shown the main ideas behind the minimum spanning tree for weighted graphs. Now let's see how to go about creating the algorithm for this process.

Storing the Edges

The key activities in the example were tracking the category for each city and listing the costs of links between pairs of cities. You decided where to build the next link (edge) and add a new city (vertex) by selecting the minimum cost link.

A collection of weighted edges in which you repeatedly select the minimum value suggests a priority queue or sorted list or minimum heap as an appropriate data structure. These are all efficient ways to handle the selection of the minimum cost edge, but you should look at the other operations that need to be done, too. You certainly need to remove the minimum cost edge from the collection, but you also need to either remove nonminimum edges that connect vertices already in the MST or discard them when they are selected as the minimum cost edge. Of course, you also must insert all the edges in the structure as they are discovered.

Inserting into a priority queue based on a sorted array is an $O(N)$ operation due to shifting the values in the array (or finding the insertion point in a list).

Inserting into heap is an $O(\log N)$ operation. For removing the minimum value, priority queues and sorted arrays need $O(1)$ time (if the minimum is kept at the end of the array so that no shifting is required) while the heap takes $O(\log N)$.

Let's assume the graph has V vertices and E edges. You only discover those values as the graph is explored, but you can give them names for the analysis. During the process, you will insert E edges into the collection and also remove at least $V - 1$ of them, perhaps discarding those that link two vertices already in the MST. You know that E must be greater than or equal to $V - 1$, or the graph must have more than one connected component. Let's summarize the costs of the various structures in [Table 15-1](#). Because we need to analyze the costs for edges and vertices, we use $O(E)$ and $O(V)$ instead of $O(N)$.

Table 15-1 *Efficiency of Different Edge Storage Schemes*

Structure	E Inserts	V - 1 Removals	Overall
Priority queue	$E \times O(E)$	$(V - 1) \times O(1)$	$O(E^2)$
Sorted list	$E \times O(E)$	$(V - 1) \times O(1)$	$O(E^2)$
Minimum heap	$E \times O(\log E)$	$(V - 1) \times O(\log E)$	$O(E \times \log E)$

In the overall costs, the insertion costs dominate. The removals are much less costly for the priority queue and sorted list, but the inserts grow as the square of the number of edges. For the heap, both insertion and removal grow logarithmically. You can simplify and use E only in the overall cost because it is bigger than V (in all but the simplest graphs). That makes it clear that heaps are the most efficient because $E \times \log E < E^2$ as E grows large.

Instead of a list or array, you use a minimum heap, as described in [Chapter 13](#), “[Heaps](#).” Because heaps don't offer an efficient way to delete items other than the minimum valued one, you simply remove the minimum edges until you find one that connects to a fringe vertex.

Outline of the Algorithm

Let's restate the algorithm in graph terms (as opposed to linking city terms).

Start with a vertex (any one will do) and put it in the tree (a subgraph). Then repeatedly do the following:

1. For each outgoing edge from the newest vertex, if it goes to a vertex already in the tree, “prune” (discard) it. Otherwise, put the edge in the minimum heap.

2. Pick the edge with the lowest weight from the heap until either
 - a. the heap is empty, or
 - b. you reach an edge with one vertex out of the tree.
3. If the heap was emptied in the previous step, then you've found the full tree for the connected component. Otherwise, add the edge to the tree, and note the destination vertex as the newest in the tree.

Repeat these steps until all the vertices are in the tree or the heap becomes empty. At that point, you're done.

In step 1, *newest* means most recently installed in the tree. The edges for this step can be found in the adjacency matrix (or perhaps computed if the graph edges have yet to be discovered). In step 2, edges with one vertex out of the tree are the edges from finalized vertices to fringe vertices.

Implementing Weights: To Infinity and Beyond

When we introduced graphs in [Chapter 14](#), we described how the adjacent vertices could be represented either as an array or as lists. When edges have weights, you need to consider the implementation options again.

Typically, weights can be any number, perhaps including floating-point values. That provides maximum flexibility in representing the weights for a wide range of problems. If you use a list of adjacent vertices, you simply add a field to each link in that list to hold the weight associated with the edge going to the vertex. In adjacency matrices, you used numbers to represent adjacent versus nonadjacent vertices (1 versus 0). Could you just put the edge weight as the number in that array cell? In other words, could `adjMat[j, k]` hold the weight of the edge between vertex indices `j` and `k`? The answer is not quite as simple as it might appear.

You need to distinguish adjacent and nonadjacent vertices. If the weights are never allowed to be zero, you could continue using zeros to represent nonadjacent vertices. That might not seem like much of a constraint, but in practice, it can be quite limiting. There are many applications in which having an edge with zero weight is useful. Sometimes even having negative weights is useful and avoiding accidentally setting a weight to zero would be very inconvenient. In this chapter, however, we assume that edge weights are zero or positive.

If you can't use zeros to represent nonadjacent vertices, what can you do? One way to address this limitation is to store a value that represents positive infinity as the edge weight for nonadjacent vertices. This allows zero weight edges and negative weights, if needed. Infinite weight is something like being infinitely distant—or unconnected. How do you represent infinity in a finite amount of memory? There are several ways, but like the way bounds and the query circle radius were managed in [Chapter 12, “Spatial Data Structures,”](#) you can use the convenient mechanism Python offers. In the `math` module, a constant called `inf` is defined that behaves like positive infinity (`+[inf]`). If it is compared with any other integer or floating-point number, it will always be greater. The only number it is equal to is itself.

Using positive infinity to represent nonadjacent vertices means you would fill the initial adjacency matrix with that value for empty graphs. Sometimes this is called a **weighted adjacency matrix** because all cells have edge weights, but only the finite ones are considered adjacent. Alternatively, in a hash table representation of the adjacency matrix, any key not in the hash table would have a weight of positive infinity, and only the adjacent vertex pairs would have finite weights stored. In this book, we implement the weighted graph class using a hash table.

Python Code

We don't have to change much in the implementation of the `WeightedGraph` class compared to the `Graph` class described in [Chapter 14](#). Only the edges have changed. [Listing 15-1](#) shows the revised parts.

Listing 15-1 *The Basic WeightedGraph Class*

```
from project_13_2_solution import Heap # Minimum heap
import math

class Vertex(object): # A vertex in a graph
... # same as for Graph

class WeightedGraph(object): # A graph containing vertices and edges
    def __init__(self): # with weights.
        self._vertices = [] # A list/array of vertices
        self._adjMat = {} # Hash table maps vertex pairs to weight
... # skipping shared definitions with Graph
```

```

def addEdge(                                # Add edge of weight w between two
    self, A, B, w):                        # vertices A & B
    self.validIndex(A)                    # Check that vertex A is valid
    self.validIndex(B)                    # Check that vertex B is valid
    if A == B:                            # If vertices are the same
        raise ValueError                  # raise exception
    self._adjMat[A, B] = w                # Add edge in one direction and
    self._adjMat[B, A] = w                # the reverse direction

def hasEdge(self, A, B): # Check for edge between vertices A & B
    return ((A, B) in self._adjMat and # If vertex tuple in adjMat
            self._adjMat[A, B] < math.inf) # and has finite weight

def edgeWeight(self, A, B): # Get edge weight between vertices
    self.validIndex(A)        # Check that vertex A is valid
    self.validIndex(B)        # Check that vertex B is valid
    return (                   # If vertex tuple in adjMat, return
        self._adjMat[A, B] if (A, B) in self._adjMat
        else math.inf)        # the weight stored there otherwise +∞

```

The weighted graph needs a `Heap` data structure, specifically the minimum (or ascending) heap from Programming Project 13.2. We also need the `math` module for the positive infinity constant. The `Vertex` class is the same because the changes apply only to the edges of the graph. The constructor is included in [Listing 15-1](#) even though it is identical to that of the `Graph`.

The first method that needs a new definition is the one for adding edges. It must now take a third parameter for the weight of the new edge. As before, `addEdge()` starts by validating the vertex indices, `A` and `B`, passed by the caller and verifies that the edge links distinct vertices. Then it stores the weight, `w`, in the `_adjMat` cell for the vertex pair in both directions. In the unweighted `Graph`, it just stored a 1 in those cells.

The next method, `hasEdge()`, changes to first test for the presence of the vertex pair among the keys of the adjacency matrix (instead of using Python's `get()` method for hash tables to supply a default value when the key is missing). It also compares any value stored in the matrix with `math.inf`, positive infinity. That comparison might not be needed in many applications because it will make a difference only if the caller stores `math.inf` as the weight of some edge. That might be done, for example, to remove an edge from the graph. If the comparison were left out of `hasEdge()`, however, then the method would return `True` for edges with infinite weight.

The final method, `edgeWeight()`, is new. We need a method that returns the weight of a given edge. It checks for valid vertex indices and then returns the weight stored in the adjacency matrix if the vertex pair is a key in the adjacency matrix. Otherwise, it returns positive infinity.

That's all that needs to be changed in the basic representation. A few other changes like including the weights in the output of the `print()` method are helpful. Now we can focus on the algorithms that use weights.

The Weighted Minimum Spanning Tree Algorithm

To find the spanning tree that minimizes the total edge weight, we can set up data structures similar to those used for the unweighted graph. [Listing 15-2](#) shows the code. After validating the starting index, `minimumSpanningTree()` creates an empty subgraph called `tree` to hold the result. It then builds an array to map vertex indices from the graph to those in the tree, `vMap`. We need that because we will be adding vertices in a different order to the tree as the algorithm discovers the edges with lowest weight. It also serves to identify which vertices have been copied (mapped) into the `tree`.

Next, an empty heap is constructed to keep the edges in partially sorted order so that it's quick to get the lowest weight edge. The items we insert on the heap of `edges` are tuples of an edge—a vertex pair—and its weight. We use the `key` parameter of the `Heap` class to specify the function that extracts the weight from each tuple. The `weight()` function is shown at the end of [Listing 15-2](#). It returns the second element of the tuple.

Listing 15-2 *The `minimumSpanningTree()` Method of `WeightedGraph`*

```
class WeightedGraph(object): # A graph containing vertices and edges
...

    def minimumSpanningTree( # Compute a spanning tree minimizing edge
        self, n):           # weight starting at vertex n
        self.validIndex(n)   # Check that vertex n is valid
        tree = WeightedGraph() # Initial MST is an empty weighted graph
        nVerts = self.nVertices() # Number of vertices
        vMap = [None] * nVerts # Array to map vertex indices into MST
        edges = Heap(         # Use min heap for explored edges
            key=weight,        # Store (A, B) vertex pair & weight in
            descending=False)  # each heap item
        vMap[n] = 0           # Map start vertex into MST
```

```

tree.addVertex(self.getVertex(n)) # Copy vertex n into MST
while tree.nVertices() < nVerts: # Loop until all verts mapped
    for vertex in self.adjacentVertices(n): # For all adjacent
        if not vMap[vertex]: # vertices that are not mapped,
            edges.insert( # put weighted edges in heap
                ((n, vertex), self.edgeWeight(n, vertex)))
    edge, w = ( # Get first edge and weight, if one exists
        (None, 0) if edges.isEmpty() else edges.remove())
    while (not edges.isEmpty() and # While there are more edges
        vMap[edge[1]] is not None): # and current edge in MST,
        edge, w = edges.remove() # go on to next edge
    if (edge is None or # If we didn't find an edge or it goes
        vMap[edge[1]] is not None): # to a mapped vertex
        break # there are no more edges to be added
    n = edge[1] # Otherwise get new vertex and
    vMap[n] = tree.nVertices() # map it into MST
    tree.addVertex(self.getVertex(n)) # copy it into MST
    tree.addEdge( # Add weighted edge to MST mapping
        vMap[edge[0]], vMap[edge[1]], w) # vertex indices
return tree # Return the minimum spanning tree

def weight(edge): return edge[1] # Get weight from edge tuple in heap

```

With the output subgraph (`tree`), the vertex map, and the edges heap built, the main part of the algorithm begins. Initially we map the starting vertex in the input graph, `n`, to vertex 0 in the output tree and add that vertex object to the tree. If the input graph happened to be a single vertex graph, that's all that would be needed because there can be no edges in the output tree.

The main `while` loop iterates until the output `tree` has as many vertices as the input graph. You'll see what happens if the graph has more than one connected component a little later. Inside the loop, we begin with the start vertex, `n`, find all its adjacent vertices, and look to see if each adjacent vertex has been mapped into the `tree`. At the start, the only mapped vertex is the initial `n`, but on subsequent passes through the loop, `n` will be the last vertex added to the output tree. That's why we must check every adjacent vertex using the `vMap` array.

Adjacent vertices that have *not* been mapped mean that the corresponding edge leads to the fringe and should be inserted in the heap. The call to `edges.insert()` puts the tuple of the edge and its edge weight in the heap. The heap knows where to place it according to the `weight` key defined when it was constructed.

Next, the method takes the first edge out of the heap or sets `edge` to `None` if the `edges` heap is empty. The inner `while` loop checks whether this `edge` connects to a vertex that is already mapped into the output tree, by looking at the value of `vMap[edge[1]]`, the mapping for the adjacent vertex. How do we know to examine the vertex at `edge[1]` instead of the one at `edge[0]`? The reason is that we add edges to the `edges` heap only where the second vertex is on the fringe and the first vertex is already in the finalized tree.

If the `edge` goes to a vertex already in the tree and there are more edges in the heap, we take the next lowest weight edge from the heap by calling `edges.remove()` in the inner loop. After the inner loop exits, if no edge was found (`edge` is `None`) or we only found edges connecting within the `tree` (`vMap[edge[1]]` is not `None`), then we've run out of edges to follow in expanding the tree. In that case, it's time to break out of the outer `while` loop because we have built the minimum spanning tree of the connected component containing the start vertex. The returned `tree` will have fewer vertices than the input graph.

After verifying that we did find an edge leading to an unmapped vertex, we set `n` to be that new vertex, `edge[1]`, map it into the output tree, add the vertex to the tree, and add the edge that led to it. Each of vertices of in the `edge` must be mapped to their new indices in the output tree using `vMap`. The next pass through the main `while` loop will explore edges extending from `n`.

When control exits from the main `while` loop, the minimum spanning tree is complete (for the connected component that includes the starting vertex). As you can see, adding weights to the minimum spanning tree algorithm makes it more complicated than the depth-first traversal that was used for the unweighted graph. Having the heap and vertex mapping array available, however, reduces the complexity of the code.

The WeightedGraph Visualization tool can display each of the steps in this algorithm. Try creating a simple graph, selecting a starting vertex, and using the step button, **▶**, to see how each operation happens.

Figure 15-9 shows the WeightedGraph Visualization tool during the computation of the minimum spanning tree of the graph shown in Figures 15-1, 15-2, and 15-3. At this point in the processing, vertices A, B, C, and D have been added to the output tree. There are a lot of data structures on the display, including

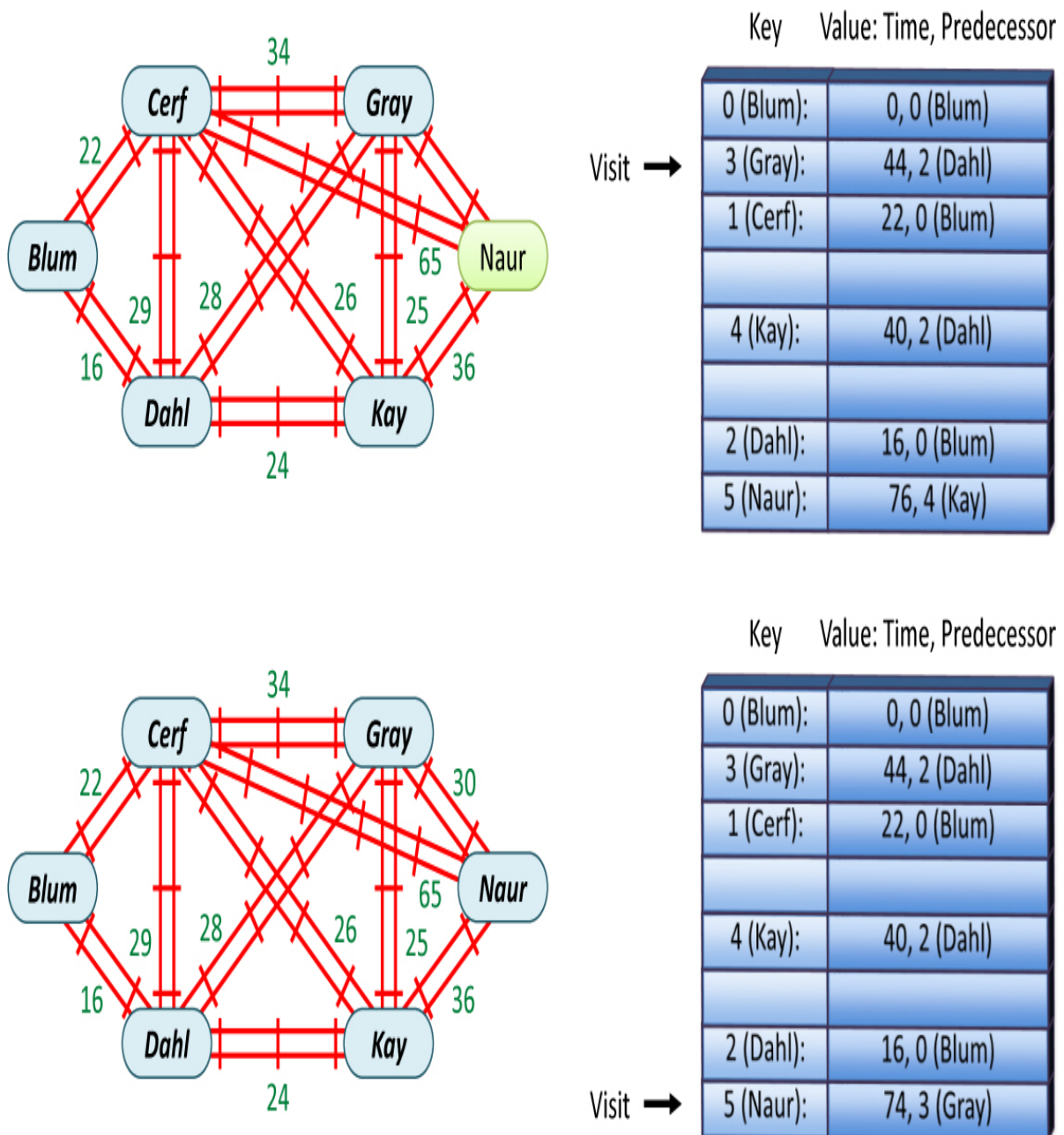


Figure 15-9 *The Visualization tool computing a minimum spanning tree*

- The array for the input graph's `_vertices` (at the upper right)
- The hash table for the input graph's `_adjMat`, adjacency matrix (which is collapsed in Figure 15-9 to show other variables)
- The `vMap` array mapping input vertices to their index in the output tree (at the upper right next to the `_vertices` array)

- The output minimum spanning `tree` (shown by a curved arrow starting at the lower left pointing to the first vertex, highlight rings on the vertices that have been mapped into the tree, and thicker, highlighted edges that have been added to tree)
- The `edges` heap shown as a simple, ordered array at the bottom instead of a heap tree (Showing the heap as a tree would take more room, and placing the items in standard heap ordering would make their relationships less clear.)

The quantity of data and their relationships are complex. Try stepping through the algorithm, watching the incremental changes to the different structures, and predicting what will change at each step before selecting the button to see the outcome.

The Shortest-Path Problem

Perhaps the most commonly encountered problem associated with weighted graphs is that of finding the shortest (lowest-weight) path between two given vertices. The solution to this problem is applicable to a wide variety of real-world situations: planning travel routes, laying out integrated circuits, project scheduling, and more. It is a more complex problem than we've seen before, so let's start by looking at a (somewhat) real-world scenario in the same mythical country of Turala.

Travel by Rail

This time you're concerned with railroads rather than network connections. While Turala was a little behind the times in terms of communication infrastructure, the country has a wonderful rail system. You want to find the fastest route from one city to another to plan for a race across the country.

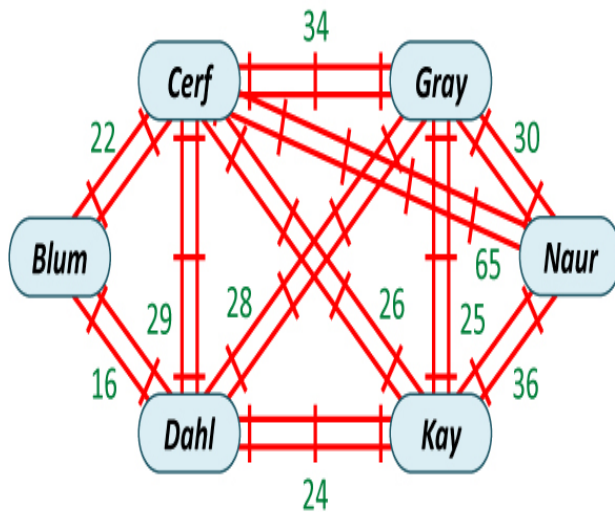
Finding the fastest route is one variant of the **shortest-path problem**. In this situation you're interested in the shortest travel times, but you could look for the shortest-distance or lowest-fare route. In weighted graphs, *shortest* doesn't necessarily mean shortest in terms of time or distance; it can also mean cheapest, lowest emission, or best route by some other measure.

The time to travel between any two cities by train can vary. That's due to the track conditions and changing weather; heavy rain, snow, or fog means the

trains travel slower. The railroads post the travel times daily, but they only post them at the station for the trains leaving from there. Your team will have to get the conditions on the day of the race.

Possible Routes

To plan your route, you need to know what options are available. The first step is to outline the possible train rides your team might take. You would likely construct a map of the cities in graph form like the one at the left of [Figure 15-10](#). There are several possible routes between any two cities. For example, to take the train from Blum to Kay, you could go through Dahl, or you could go through Cerf and Gray, or through Dahl and Gray, or you could take several other routes.



Key Value: Time, Predecessor

0 (Blum):	0, 0 (Blum)
3 (Gray):	44, 2 (Dahl)
1 (Cerf):	22, 0 (Blum)
4 (Kay):	40, 2 (Dahl)
2 (Dahl):	16, 0 (Blum)
5 (Naur):	74, 3 (Gray)

Reconstruct the final path from the predecessors in the cost table

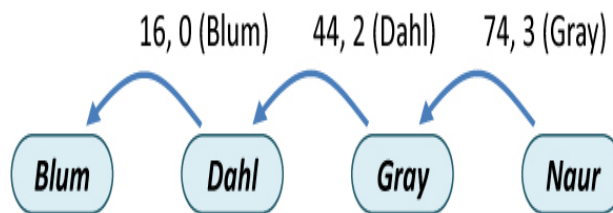


Figure 15-10 *Train travel in Turala*

Note the graph in [Figure 15-10](#) doesn't have travel times (or fares or schedule). All you know at this point is that the race will start in Blum, so that's why it's highlighted.

A Directed, Weighted Graph

The travel times between cities can be different depending on the direction of travel. Trains going uphill might proceed more slowly than those going

downhill. Planes flying against the jet stream have lower ground speed than those flying with it. To model differences like these, we use directed graphs where the weight for traveling from vertex J to vertex K is different than the weight from K to J. The Turalan railroad has only single-track lines, so you can go in only one direction between any two cities at any point in time. In your solution, you will only note the travel times going away from Blum because you don't need to plan a round trip. In the case of finding a lowest-fare route, you could use an undirected graph if the fare was the same in both directions.

Dijkstra's Algorithm

The solution we show for the shortest-path problem is called Dijkstra's algorithm, after Edsger Dijkstra, who first described it in 1959. Interestingly, the method not only solves for the shortest path to a destination city, but it can also solve for the shortest path to any other destination, as you shall see.

Agents and Train Rides

On the day of the race, you learn that the finish line is in Naur. To see how Dijkstra's algorithm works, your team is going to operate the way a computer does, looking at one piece of information at a time, so we assume that you are similarly unable to see the big picture (as in the preceding section). That means your team needs to send agents to stations like the surveying teams for the network routing.

At each city, the stationmasters can tell you how long it will take to travel to the other cities that you can reach directly (that is, in a single ride, without passing through another city). Alas, they cannot tell you the times to cities further than one ride away. You keep a notebook, like the one at the right of [Figure 15-10](#), with a row for each city. You hope to end up with rows filled in with the shortest time from your starting point to that city (plus a little more information that you will need at the end).

The First Agent: In Blum

Eventually, you're going to place agents in every city (at least those needed to get to Naur). These agents must obtain information about travel times to other cities. You yourself are the agent in Blum.

All the stationmaster in Blum can tell you is that today it will take 22 minutes to get to Cerf and 16 minutes to get to Dahl. You write this information in your notebook, as shown in [Table 15-2](#).

Table 15-2 *Step 1: Notebook with Information from the Agent at Blum*

To	Time	Via
Cerf	22	Blum
Dahl	16	Blum

The table lists all the cities for which you have some information about the travel time. The Via column records what city you came from to get that time. You'll see later why this is good to know. What do you do now? Here's the rule you follow:

Rule

Always send an agent to the unvisited city whose overall route from the starting point (Blum) is the shortest.

Notice that this is not quite the same rule as that used in the minimum spanning tree problem (the network installation). There, you picked the least expensive *single link* (edge) from the connected cities to an unconnected city. Here, you pick the least expensive *total route* from Blum to a city with no agent. At this particular point in your planning, these two approaches amount to the same thing because all known routes from Blum consist of only one edge. As you send agents to more cities, however, the routes from Blum will become the sum of several direct edges.

The Second Agent: In Dahl

The shortest route from Blum is to Dahl, at 16 minutes. So, you send half your team to Dahl, where one will act as the agent and others will be ready for later tasks. When they arrive, they message you that the Dahl stationmaster says today's trains will take 29 minutes to reach Cerf, 28 minutes to reach Gray, 24 minutes to reach Kay, and 15 minutes to return to Blum. That last travel time confuses your agents, until they realize the stationmaster doesn't know where they came from and is just trying to be helpful.

Now you can update your notebook. Adding up the time you already spent getting to Dahl with the times to reach Gray and Kay, you can make two new entries in the notebook as shown in [Table 15-3](#). You now know that you can reach Gray in 44 minutes and Kay in 40 minutes from Blum. You also received information about Cerf. If you travel to Cerf via Dahl, it takes $16 + 29 = 45$ minutes. That's longer than the direct travel from Blum, so there's no need to change the entry for Cerf.

Table 15-3 *Step 2: Notebook with Information from the Agents at Dahl*

To	Time	Via
Cerf	22	Blum
Dahl	16	Blum
Gray	44	Dahl
Kay	40	Dahl

It helps to see how this information looks in graph form. [Figure 15-11](#) shows the weights on the routes (edges) that have been learned so far. You have agents in Blum and Dahl, and a quick look among the other cities in your notebook shows that Cerf is the next shortest route. Following the rule, you and the other agents in Blum go to Cerf to learn what conditions are like there. That's what the "Visit" pointer is showing; it's time to visit Cerf.

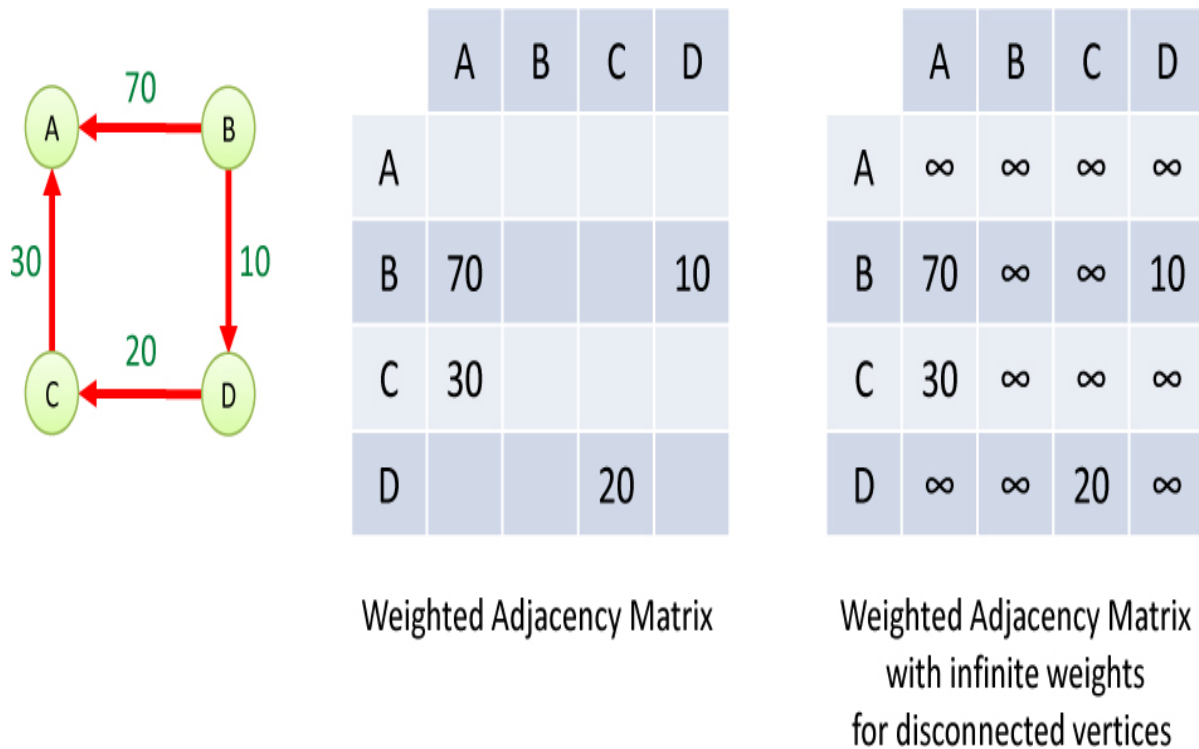


Figure 15-11 *Following step 2 in the shortest-path algorithm.*

The table on the right of [Figure 15-11](#) shows the same information that's in your notebook but in a different format. The cities are in a different order, it includes the start city of Blum, and there are some extra numbers along with the city names. The reasons for those are explained shortly.

After you've placed agents in a city, you can be sure that the route taken by them to get to that city is the fastest route. Why? Consider the present case. If there were a faster route than the direct one from Blum to Cerf, it would need to go through some other city. The only other way out of Blum by train is to Dahl, which you explored first. You already found that the Blum–Dahl–Cerf route takes more time than the direct route to Cerf. If there were a route via Gray, Kay, or some other city back to Cerf, it would have to take at least 40 minutes (the time needed to reach Kay), so there's no faster way via Dahl. Hence, you conclude with certainty that you know the shortest time to all the visited cities (Blum, Dahl, and now Cerf).

Based on this analysis, you decide that from now on you won't need to update the entries for the time from Blum to Cerf or Blum to Dahl. You won't be able to find faster routes, so you can cross them off the list.

Three Kinds of Cities

As in the minimum spanning tree algorithm, the cities for the shortest-path algorithm are divided into three categories:

1. Cities in which you've installed an agent (for which you've already found the shortest path).
2. Cities with known travel times from cities with an agent; they're on the fringe.
3. Unknown cities.

At the beginning of step 2, Blum and Dahl are category 1 cities because they have agents there. Based on the travel times learned in Dahl, you sent agents to Cerf, moving that city to category 1. Those three cities form a tree consisting of paths that all begin at the starting vertex (Blum) and that each end on a different destination vertex. This is not the same tree, of course, as a minimum spanning tree. It's the shortest-path tree because you've concluded you now know the fastest routes to each city.

Two other cities have no agents, but you know some of the times to reach them because you have agents in adjacent category 1 cities. You know the time to go from Blum to Gray is at most 44 minutes and to Kay is at most 40 minutes. The information in your notebook means that Gray and Kay are category 2 (fringe) cities.

You don't know anything yet about Naur; it's an "unknown" city (from the perspective of this algorithm). As in the minimum spanning tree algorithm, Dijkstra's shortest-path algorithm moves cities from the unknown category to the fringe category, and from the fringe category to the tree, as it goes along.

The Agents in Cerf

With you and your other agents now in Cerf, you can cross out Cerf and Dahl in your list and get information from the stationmaster there. Cerf is a busy station because five rail lines connect to it. Your agents learn today's trains take 34 minutes to Gray, 65 minutes to Naur, 26 minutes to Kay, 24 minutes to Dahl, and 18 minutes to Blum. You add the times to your notebook, for the unvisited cities to get [Table 15-4](#).

Table 15-4 *Step 3: Notebook with Information from the Agents at Cerf*

To	Time	Via
Cerf	22	Blum
Dahl	16	Blum
Gray	44	Dahl
Kay	40	Dahl
Naur	87	Cerf

The Blum–Cerf–Gray route takes $22 + 34 = 56$ minutes. That’s more than the Blum–Dahl–Gray route of 44 minutes, so the entry from before remains. Similarly, Blum–Cerf–Kay takes $22 + 26 = 48$ minutes, which is slower than Blum–Dahl–Kay at 40. You do make a new entry for Naur (so it is no longer in the unknown category).

The times to return to Blum or Dahl are less than what it took to go to Cerf from those cities. Shouldn’t that affect the route plan? It might be that Cerf is higher in elevation so going back is faster, but that’s not going to help you pick the fastest route *from* Blum *to* Naur. The crossed-out entries don’t need updates.

You now have the situation shown in [Figure 15-12](#). The information learned in Cerf shows that the next city to visit is Kay with the total time from Blum being 40 minutes. It’s also clear that the fastest route to Kay is the Blum–Dahl–Kay route because you noted the predecessor of Kay was Dahl in the notes. You now have all of the cities in category 1 or category 2; none remain in category 3.

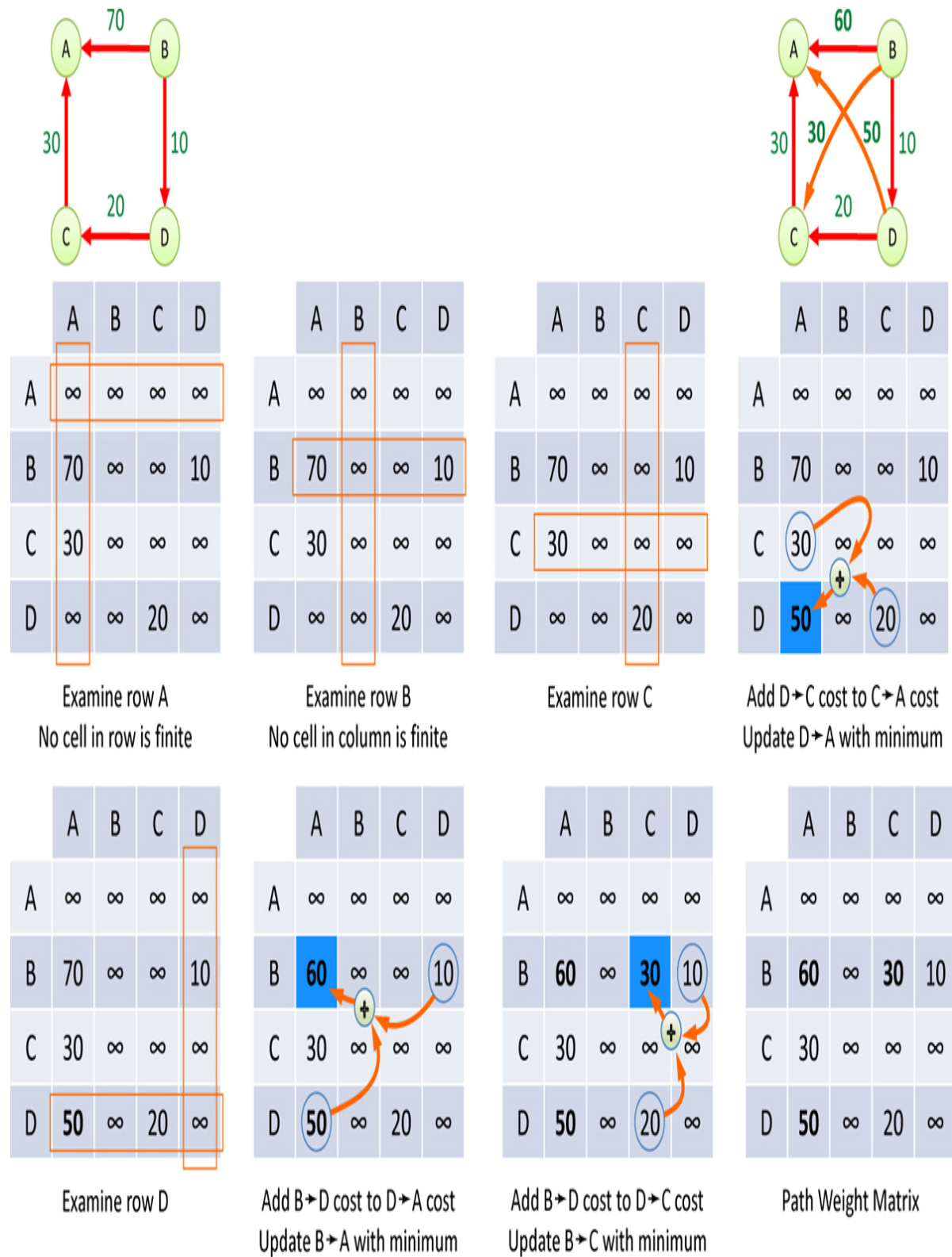


Figure 15-12 Following step 3 in the shortest-path algorithm.

Because you have Naur in your notebook, can you stop sending agents out? Some members of the team are eager to finish the surveying. You do have one route to Naur, but it's not certain yet that you know the very fastest route. You tell your team in Dahl to split up, move half to Kay, and press on.

The Agents in Kay

When you have agents in Kay, they quickly report the following travel times: Gray 25 minutes, Naur 36 minutes, Cerf 29 minutes, and Dahl 24 minutes. Those last two are cities you have already crossed off, so only the first two matter. Adding the times to the shortest time to get to Kay of 40 minutes (via Dahl) means you could get to Gray in 65 minutes and Naur in 76 minutes. You already know a route to Gray that takes 44 minutes (via Dahl), so the new route is no good. The route to Naur, however, is shorter. (It was worth pressing on!) You update your notebook to show what's in [Table 15-5](#).

Table 15-5 *Step 4: Information from the Agents at Kay*

To	Time	Via
Cerf	22	Blum
Dahl	16	Blum
Gray	44	Dahl
Kay	40	Dahl
Naur	87 76	Cerf Kay

You are faced with another choice of ending the process with the newer, better route to Naur. You trust in the process, however, and decide to continue. The shortest route to an unvisited city is now to Gray (via Dahl) at 44 minutes. This corresponds to the upper graph of [Figure 15-13](#).

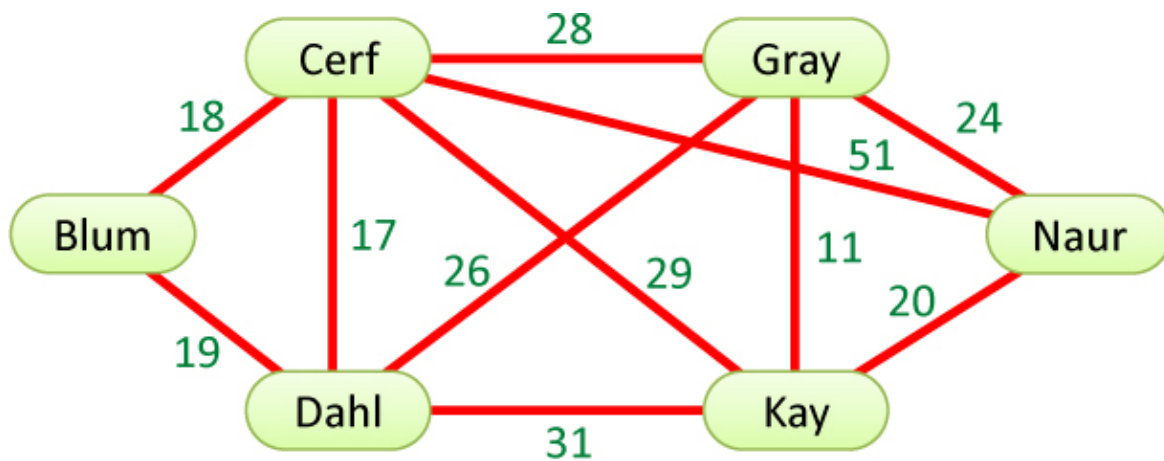


Figure 15-13 *Following steps 4 and 5 in the shortest-path algorithm.*

You're closing in on the goal. You send the remaining team in Dahl to Gray because that was the fastest way to get to Gray based on the notes. (A sharp observer might note that you could get agents from Kay there faster, but that's not really part of the computer algorithm, which simply has to choose which vertex to explore next.)

The Last Agents in Gray

Your (now very experienced) agents report the only route that matters from Gray: it will take 30 minutes to reach Naur from there. Because it takes 44 minutes to get to Gray (via Dahl), you now could get to Naur in 74 minutes. You make another revision to your notebook and end up with the graph and table shown at the bottom of [Figure 15-13](#). You send an agent to Naur, just to confirm the time.

When there's an agent in every city, you know the best times from Blum to every other city. You're done. With no further calculations, the last entries in your notebook show today's fastest routes from Blum to all other visited cities. Your team now has the optimal route for the Blum to Naur race. That route plus the other fastest routes form a tree rooted at the starting point (not to be confused with the minimum spanning tree).

Note that you don't have the fastest route to any cities not yet visited when you stop updating the notebook. If Turala had a dozen more cities, you wouldn't know for sure how long it takes to get to them. To get all the fastest routes, you would need to visit every city. You have, however, found the fastest way to get from Blum to Naur and know that any other team who simply chose the direct train from Cerf to Naur would lose the race!

This narrative has demonstrated the essentials of Dijkstra's algorithm. On the surface, it seems more like the breadth-first search than the depth-first one, expanding the shortest-known path away from the starting point. The key points are

- Each time you send agents to a new city, you use the new information provided by them to revise your list of times. Only the shortest route (that you know about) from the starting point to a given city is retained, including the previously visited city that achieved that time.
- You always send your next agents to the unvisited city that is closest (in time) to the starting point. That is different from the fastest single train trip (edge) from any city with an agent, as was selected in the minimum spanning tree.

In the terms of graphs, Dijkstra's algorithm adds one vertex to its output subgraph on each iteration. The vertex added has the lowest total edge weight on the path back to the starting vertex. Because that path back is unique (each vertex has a unique preceding vertex), the subgraph is always a tree. If there are multiple, equal weight paths to a vertex, it will choose a path with fewer edges due to the breadth-first style exploration.

Finding Shortest Paths Using the Visualization Tool

Let's see how Dijkstra's algorithm looks using the WeightedGraph Visualization tool. If you happen to have the tool where you can launch it using the command line, you can run

```
python3 WeightedGraph.py -TuralaSP
```

This command creates the graph using two-letter prefixes for the city names and the weights shown in the previous examples. The vertex positions are random, so if you want the graph in the tool to look like the examples, you will probably have to move them around to look something like [Figure 15-14](#).



Figure 15-14 *The railroad scenario in WeightedGraph Visualization tool*

Two vertices must be selected to calculate a shortest path: a start and an end vertex. The Visualization tool lets you select the start vertex with first mouse button, just as with the minimum spanning tree. You select the end vertex by holding the Shift key or pressing a different mouse button. The vertices are highlighted with rings like those in [Figure 15-14](#) for Bl(um) and Na(ur).

Note that the visualization tool shows the graph as undirected. Dijkstra's algorithm works on both kinds of graphs, and the results are the same (assuming only positive weights and the weights are those that are needed for the final direction of travel). Showing only one edge between a pair of vertices keeps the display less cluttered.

After the graph is configured and the endpoints selected, the Shortest Path operation is enabled. Try pressing the button and stepping through the process. We look at some key steps next, focusing on the “notebook” used for tracking the shortest paths and the visited vertices before looking at the details of the implementation.

The Costs Notebook

The Visualization tool draws the notebook entries a little differently than the preceding figures. At the start of the shortest-path algorithm, it creates a structure at the bottom of the display called `costs` with a single entry for Bl(um), as in [Figure 15-15](#). The top of the entry is the vertex name along with its matching-colored background. The bottom of the entry is the total time (weight, distance, cost, and so on) to reach that vertex, along with the vertex visited just before. These are shown as tuple like `(0, 'B1')`, where the first

component is the total time and the second is the previous or *parent vertex*. For this starting vertex, there is no previous one, so it's marked as being Bl(um).



Figure 15-15 *The first entry in `costs` for the shortest path to Naur*

In the upper right of the display, a `visited` array tracks which vertices have been finalized with their shortest path (put in category 1). The 1 in the cell next to Bl(um) indicates that it—and it alone—falls in this category at this stage. The total time for category 1 vertices in the `costs` structure is that for the final, shortest path.

The display has many different arrows indicating variables used in different parts of the program, which can be confusing. Several of them show the same information but in different ways. For example, the `start` and `end` variables indicate the start and end vertices in both the `_vertices` array and the graph layout. The `nextVert` arrow shows the next vertex to be processed in both the `costs` structure and the graph layout. Sometimes these arrows and other display elements overlap each other, making them harder to read.

When the algorithm chooses the second city to visit, Da(hl), the situation looks like [Figure 15-16](#). The `costs` structure now has three entries, showing the shortest paths found so far for the three vertices, just like the notebook in [Table 15-2](#). All of them have Bl(um) as the preceding vertex. The `visited` array shows that only Bl(um) and Da(hl) have been visited, and hence are in category 1 with their final shortest paths.



Figure 15-16 *After selecting Da(hl) to visit in finding the shortest path to Naur*

The figure also collapses the adjacency matrix to show the variables `cost` and `pathCost`, which might otherwise be hidden. The edge weights remain visible in the graph layout, so the adjacency matrix information is still available.

The process continues visiting cities and updating the structures. After skipping over the visit to Ce(rf), [Figure 15-17](#) shows the contents of the `costs` and `visited` structures when Ka(y) is visited. It shows that both Gr(ay) and Ka(y) were reached from Da(hl). Ka(y) has a shorter total path cost of 40 compared to Gr(ay)'s cost of 44, so it is chosen as the next vertex to visit. The `visited` array shows that Ka(y) has now been put in category 1. Ce(rf) was visited in the previous stage, but the path through Da(hl) took less time.

Both Gr(ay) and Na(ur) are unvisited at this stage, but you do have estimated times to reach them of 44 and 87, respectively. Their parent vertices are different because they were reached along different paths.